Task 5 – Same Screen, Different Event-Handling Styles: Currency Converter (7 Marks, K5)

> Run each program yourself and attach your own screenshots, each linked to the requirement or test it shows. Items marked `<fill>` need your own observations or measurements.

Task 5.1 – Four implementations

All four share one conversion method (`Convert.run`) so that only the *event-handling style* differs. Window closing is handled with a `WindowAdapter` in each. Full code is in `ConverterStyles.java`:

```
import java.awt.FlowLayout;
import java.awt.event.*;
import javax.swing.*;

/** Style (a): frame implements ActionListener. */
class StyleA extends JFrame implements ActionListener {
    JTextField amt = new JTextField(8); JLabel res = new JLabel(" ");
    JComboBox<String> to = new JComboBox<>(new String[]{"USD",
"EUR"});
    JButton b = new JButton("Convert");
    StyleA() {
        super("A: frame is listener"); setLayout(new FlowLayout());
        add(amt); add(to); add(b); add(res);
        b.addActionListener(this);                        // 'this'
handles every source
        addWindowListener(new WindowAdapter() { @Override public void
windowClosing(WindowEvent e) { dispose(); } });
        pack();
    }
    @Override public void actionPerformed(ActionEvent e)
{ Convert.run(amt, to, res); }
}

/** Style (b): named inner class. */
class StyleB extends JFrame {
    JTextField amt = new JTextField(8); JLabel res = new JLabel(" ");
    JComboBox<String> to = new JComboBox<>(new String[]{"USD",
"EUR"});
    JButton b = new JButton("Convert");
    class ConvertListener implements ActionListener {   // can use the
frame's fields directly
        @Override public void actionPerformed(ActionEvent e)
{ Convert.run(amt, to, res); }
    }
    StyleB() {
```

```
        super("B: named inner class"); setLayout(new FlowLayout());
        add(amt); add(to); add(b); add(res);
        b.addActionListener(new ConvertListener());
        addWindowListener(new WindowAdapter() { @Override public void
windowClosing(WindowEvent e) { dispose(); } });
        pack();
    }
}

/** Style (c): anonymous inner class. */
class StyleC extends JFrame {
    JTextField amt = new JTextField(8); JLabel res = new JLabel(" ");
    JComboBox<String> to = new JComboBox<>(new String[]{"USD",
"EUR"});
    JButton b = new JButton("Convert");
    StyleC() {
        super("C: anonymous class"); setLayout(new FlowLayout());
        add(amt); add(to); add(b); add(res);
        b.addActionListener(new ActionListener() {
            @Override public void actionPerformed(ActionEvent e)
{ Convert.run(amt, to, res); }
        });
        addWindowListener(new WindowAdapter() { @Override public void
windowClosing(WindowEvent e) { dispose(); } });
        pack();
    }
}

/** Style (d): lambda. */
class StyleD extends JFrame {
    JTextField amt = new JTextField(8); JLabel res = new JLabel(" ");
    JComboBox<String> to = new JComboBox<>(new String[]{"USD",
"EUR"});
    JButton b = new JButton("Convert");
    StyleD() {
        super("D: lambda"); setLayout(new FlowLayout());
        add(amt); add(to); add(b); add(res);
        b.addActionListener(e -> Convert.run(amt, to, res));
        addWindowListener(new WindowAdapter() { @Override public void
windowClosing(WindowEvent e) { dispose(); } });
        pack();
    }
}

/** Shared conversion logic. */
class Convert {
    static double rate(String c) { return c.equals("USD") ? 0.012 :
0.011; }
    static void run(JTextField amt, JComboBox<String> to, JLabel res)
{
```

```
        try {
            double v = Double.parseDouble(amt.getText().trim()) *
rate((String) to.getSelectedItem());
            res.setText(String.format("%.2f %s", v,
to.getSelectedItem()));
        } catch (NumberFormatException ex) { res.setText("Invalid
amount"); }
    }
}

public class ConverterStyles {
    public static void main(String[] a) {
        SwingUtilities.invokeLater(() -> {
            new StyleA().setVisible(true); new
StyleB().setVisible(true);
            new StyleC().setVisible(true); new
StyleD().setVisible(true);
        });
    }
}
```

Key line in each style:

Style	How the Convert button is wired
(a) Frame implements `ActionListener`	`b.addActionListener(this);` and `actionPerformed(ActionEvent e)` in the frame class
(b) Named inner class	`class ConvertListener implements ActionListener {...}` then `b.addActionListener(new ConvertListener());`
(c) Anonymous inner class	`b.addActionListener(new ActionListener() { public void actionPerformed(ActionEvent e) {...} });`
(d) Lambda	`b.addActionListener(e -> Convert.run(amt, to, res));`

Task 5.2 – Change the requirement

New requirement: also convert when Enter is pressed in the amount field, and show the result in a dialog.

Style	Change needed	Approx. extra code
(a) Frame is the listener	Add	2 lines, but the handler

Style	Change needed	Approx. extra code
	`amt.addActionListener(this);`. Because one `actionPerformed` serves every source, you may also need `if (e.getSource() == ...)` if the sources must behave differently. Add `JOptionPane.showMessageDialog(...)` in the handler.	grows an `if` per source
(b) Named inner class	Add `amt.addActionListener(new ConvertListener());` (the same class is reused). Add the dialog call in `ConvertListener`.	2 lines
(c) Anonymous class	The listener has no name, so it cannot be reused. Either copy the whole anonymous class a second time or first move it into a variable.	5–8 lines (or a refactor)
(d) Lambda	`ActionListener l = e -> {...};` `b.addActionListener(l);` `amt.addActionListener(l);` and add the dialog call.	2 lines (one variable)

Record your own line counts from the edited files `<fill>`.

Task 5.3 – Comparison

Criterion	(a) Frame implements	(b) Named inner	(c) Anonymous	(d) Lambda
Readability	Poor with many sources (one big `if/switch`)	Good: named, separate	Moderate: bulky inline	Best for one-line handlers
Access to frame's fields	Direct	Direct (inner class)	Direct	Direct (effectively

Criterion	(a) Frame implements	(b) Named inner	(c) Anonymous	(d) Lambda
				final captured variables, fields freely)
Reuse of handler	One handler for everything, hard to separate	Excellent: reusable by name	Poor: no name	Good if stored in a variable
Ease of change	Grows messy	Easy	Easy locally, copy-paste risk	Very easy
Extra class files	none	one	one per use (`Frame$1.class`)	none (uses `invokedynamic`)

For a large application: named inner classes (or separate top-level listener classes / Swing `Action` objects) for non-trivial handlers, with lambdas for one-line wiring such as `e -> controller.convert()`. The frame should not implement the listener interfaces itself because it exposes `actionPerformed` publicly and ends up as a "god" handler.

Complete the Code (lambda version, ConverterLambda.java)

TODO 1 (inside the constructor):

```
ActionListener convertAction = e -> convert();
convertButton.addActionListener(convertAction);
amount.addActionListener(convertAction);       // Enter key in the
field
```

TODO 2 (inside `convert()`):

```
try {
    double inr = Double.parseDouble(amount.getText().trim());
    double value = inr * rate((String) target.getSelectedItem());
    result.setText(String.format("%.2f %s", value,
target.getSelectedItem()));
} catch (NumberFormatException ex) {
    result.setText("Invalid amount");
}
```

Full file:

```
import java.awt.FlowLayout;
import java.awt.event.ActionListener;
import javax.swing.JButton;
import javax.swing.JComboBox;
```

```java
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JTextField;
import javax.swing.SwingUtilities;

public class ConverterLambda extends JFrame {
    private final JTextField amount = new JTextField(10);
    private final JComboBox<String> target =
            new JComboBox<>(new String[]{"USD", "EUR"});
    private final JLabel result = new JLabel(" ");

    private static double rate(String currency) {
        return currency.equals("USD") ? 0.012 : 0.011;
    }

    public ConverterLambda() {
        super("Currency Converter");
        setLayout(new FlowLayout());
        JButton convertButton = new JButton("Convert");
        add(new JLabel("INR")); add(amount); add(target);
        add(convertButton); add(result);
        // TODO 1 (completed): ONE lambda listener for the button and
the Enter key
        ActionListener convertAction = e -> convert();
        convertButton.addActionListener(convertAction);
        amount.addActionListener(convertAction);

        setDefaultCloseOperation(EXIT_ON_CLOSE);
        pack();
    }

    private void convert() {
        // TODO 2 (completed)
        try {
            double inr = Double.parseDouble(amount.getText().trim());
            double value = inr * rate((String)
target.getSelectedItem());
            result.setText(String.format("%.2f %s", value,
target.getSelectedItem()));
        } catch (NumberFormatException ex) {
            result.setText("Invalid amount");
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new
ConverterLambda().setVisible(true));
    }
}
```

Expected results (take screenshots): 1000 INR → USD shows `12.00 USD`; 1000 INR → EUR shows `11.00 EUR`; typing `abc` shows `Invalid amount`. Pressing Enter in the field behaves exactly like pressing Convert.

Alternative concept and complexity

Method reference (`this::convert`) or one shared `Action` object used by button, menu item and keyboard shortcut:

```
Action convertAction = new AbstractAction("Convert") {
    { putValue(Action.ACCELERATOR_KEY, KeyStroke.getKeyStroke("control
ENTER")); }
    public void actionPerformed(ActionEvent e) { convert(); }
};
convertButton.setAction(convertAction);
convertMenuItem.setAction(convertAction);
amount.addActionListener(convertAction);
// method-reference version (convert declared as: void
convert(ActionEvent e)):
convertButton.addActionListener(this::convert);
amount.addActionListener(this::convert);
```

(`this::convert` works because `ActionListener.actionPerformed` takes one `ActionEvent`, matching the signature of `convert(ActionEvent e)`.)

- **Number of listener objects (space):** inner/anonymous classes create one listener object per trigger (k triggers → k objects, O(k)). A method reference or a shared `Action` is one object used by all triggers, O(1). Non-capturing lambdas are also created once by the JVM.
- **Work to add a new trigger:** with a shared `Action`, a new trigger is one line (`item.setAction(convertAction)`), with text, icon, shortcut and enabled-state shared automatically (disabling the action disables every trigger). With styles (a)–(d) you must wire a handler again at each trigger and keep enabled-states in sync by hand, O(k) edits for k triggers.